EcoRouter Teleprompter | Ashwin | CS 153 | ~4 min

[0:00-0:15] OPENING

Hi, I'm Ashwin.

This is EcoRouter - an autonomous large language model agent that routes AI compute workloads to the greenest data center regions in real time, while still respecting urgency deadlines and data residency constraints.

I built this for Stanford CS 153, in the Automation and Agent Systems track, as part of the One-Person Frontier Lab project.

[0:15-1:00] Q1 - WHY DID YOU BUILD THIS?

So - why did I build EcoRouter?

As artificial intelligence scales, the real bottleneck is no longer just GPUs. It's electricity.

Training a model like Llama 3, or running large batch inference jobs, consumes a catastrophic amount of power. And the grid is not static. Solar output, wind availability, and consumer demand all shift throughout the day. That means carbon intensity - measured in grams of CO2 per kilowatt-hour - changes constantly, and it changes differently in every region.

Today, deciding where and when to run these workloads usually falls to a dedicated DevOps or FinOps team. Someone has to manually monitor energy markets, watch regional grid data, and make scheduling calls. That process is slow, expensive, and it simply does not scale with the volume of AI compute being deployed.

EcoRouter automates that entire role.

It acts as a digital twin infrastructure manager. It reads live grid telemetry, evaluates a queue of incoming AI jobs, and dispatches each workload to the lowest-carbon region it can legally and operationally reach.

That is the core insight: replace manual infrastructure scheduling with an intelligent agent.

[1:00-1:45] Q2 - HOW DOES IT WORK? (ARCHITECTURE)

Now let me walk through how the system actually works.

EcoRouter is a four-layer digital twin simulation.

Layer one is the grid simulator. It mocks four global cloud regions - US East, US West, EU Central, and AP South - and generates live, fluctuating carbon intensity readings for each one.

Layer two is the job queue. It injects realistic AI workloads - things like model training, batch image processing, and transcription pipelines. Every job carries three parameters: how many compute hours it needs, whether it is urgent or flexible, and whether it has a locality constraint - for example, GDPR requiring data to stay in the EU.

Layer three is the agent brain, in ecorouter.py. This is powered by Gemini 2.5 Flash through the modern Google GenAI SDK. Critically, it uses LLM tool-calling. The model does not follow a hard-coded set of if-else

rules. Instead, it calls structured tools - get grid carbon intensity, and assign workload - and reasons through each decision step by step.

Layer four is this Streamlit dashboard. It visualizes live telemetry, displays the pending job queue, and lets a user trigger a full optimization cycle with a single button click.

Together, these four layers form a complete, end-to-end agent system.

[1:45-2:15] Q2 - LIVE DEMO SETUP

Let me show it running.

Right now, we are looking at live simulated grid telemetry. You can see carbon intensity across all four regions. In this cycle, EU Central is the greenest - but that shifts every time we refresh the data.

Below that is the pending job queue. Notice that each job has a different profile. Some are urgent. Some are flexible. And this one here has a locality constraint locked to US East - meaning the agent cannot simply send it to the globally greenest region. That is a realistic compliance constraint, like data residency under GDPR.

I am going to click Run EcoRouter Optimization, and watch the agent work.

CLICK: Run EcoRouter Optimization button. Wait for spinner (2-5 sec).

[2:15-3:00] Q2 - LIVE DEMO RESULTS

The agent has finished routing.

It called tools to read the grid, evaluated every constraint, and recorded a dispatch decision for each job.

Look at the badges on each assignment card.

The green badge means optimal carbon routing. That is a flexible job with no locality limit, successfully sent to the greenest available region.

The amber warning badge means a rigid locality constraint was in play. The agent had to route to the required region, even though a greener option existed elsewhere. That is exactly the tradeoff a real-world scheduler faces - carbon optimization versus compliance.

Each card also shows an estimated carbon cost: grid intensity multiplied by compute hours. That gives us a concrete environmental accounting for every dispatch decision.

At the top, you can see the total estimated carbon for this entire batch.

Point at: green OPTIMAL badge, amber LOCALITY badge, Est. Total Carbon metric.

[3:00-3:20] Q2 - CLI (OPTIONAL)

The same routing engine also runs headless from the command line.

Same grid simulator, same job queue, same agent logic - just without the dashboard. That makes it straightforward to integrate into a production orchestrator like Kubernetes or a cloud batch scheduler down

the road.

[3:20-4:00] Q3 - USE CASES AND SOCIETAL IMPACT

So who would actually use this, and why does it matter?

Cloud providers and AI research labs could plug EcoRouter into their batch pipelines to reduce Scope 2 emissions without breaking service-level agreements. A flexible job - like overnight checkpoint compression or non-critical data processing - could be delayed until a greener hour, or shifted from a coal-heavy region to one running on wind and hydro.

For decentralized physical infrastructure networks, or DePIN projects, the same agent logic applies anywhere compute is geographically distributed.

For society more broadly, this is about making AI growth sustainable. We are in the middle of an intelligence revolution. The question is whether that revolution has to come with an equally massive carbon tax - or whether we can automate smarter decisions at the infrastructure layer.

EcoRouter is a step toward the second path.

Click Simulate Live Grid Shift in sidebar. Point at changing bar chart.

[4:00-4:30] Q4 - WHAT MORE WOULD YOU ADD?

If I were to keep building, three things are at the top of the list.

First, predictive grid forecasting. Right now the agent reacts to real-time data. I would add time-series models that look six to twelve hours ahead, so the agent can schedule jobs proactively rather than just routing in the moment.

Second, edge deployment. I would migrate this from a local Streamlit app to a globally distributed service on Cloudflare Workers, so it can run in production close to actual infrastructure.

Third, an A/B baseline comparison. I want to quantify exactly how many grams of CO2 EcoRouter saves versus a naive round-robin scheduler - hard numbers that prove the agent is delivering real environmental value.

The full project is open source at github.com/ashwin122705/ecorouter-agent.

Thank you for watching.

Show GitHub repo URL on screen. Hold 2 seconds. End recording.